ASSIGNMENT 4 – CO4

TASK 1 – FILE-SYSTEM DESIGN REVIEW

Operating Systems (CSA04) | 6 Marks | Bloom's Level: K5 – Evaluate

GitHub Repository: To be added after creation.

GitHub URL: To be inserted after the repository is created and verified.

1. Problem Statement and Design Objective

The proposed college server places every file belonging to 12,000 students in one flat directory. Files may use only the student's chosen name, every file uses contiguous allocation, and no access control is provided because the proposal assumes students will not inspect one another's files. The objective is to evaluate the risks and replace this organisation with a searchable, protected structure that still supports group-project sharing.

The assignment asks for a reasoned evaluation, an improved directory organisation, a privacy-and-sharing defence of no more than 100 words, and an evaluation of a cloud-service alternative such as Google Drive or OneDrive.

2. Task 1.1 – Evaluation of the Proposed Organisation

Weakness	Evaluation and consequence
1. Name clashes in one directory	Two or more students can choose the same filename, such as report.pdf or final.docx. A flat namespace cannot distinguish ownership from the name alone; users may overwrite, misidentify or retrieve another student's file.
2. Search among 12,000 students	A user or application must first identify the correct owner and then the file. If directory entries are scanned sequentially, lookup can require up to 12,000 entry checks (O(N), N = 12,000). Some file systems index large directories, but the proposal does not specify such indexing or an owner-based path.
3. File growth with contiguous allocation	Contiguous allocation needs a run of consecutive free blocks. A file that grows may not have adjacent free blocks, requiring relocation/compaction or an extent-reservation strategy. Over time, free space can become externally fragmented.
4. No protection or privacy	Without owner, group and access checks, another student may read, modify or delete personal work. The claim that students will not look at each other's files is not a security control and does not prevent accidental or malicious access.
5. Backup and recovery	A single undifferentiated directory makes ownership, retention and selective restoration harder. A mistaken deletion, overwrite, malware incident or server failure can affect many students; the proposal specifies no versioning, backup schedule or recovery test.
6. Group-project sharing	Files are stored in personal names with no controlled shared workspace. Students may exchange duplicate copies, create conflicting versions, or grant access too broadly. The proposal has no project membership list or shared-folder permission model.

Overall finding: the proposal optimises neither ownership, growth nor collaboration. It also treats privacy as a matter of trust instead of enforcing it through access-control mechanisms.

3. Task 1.2 – Improved File-System Organisation

3.1 Directory tree

Use a hierarchical directory tree so the path identifies the owner or resource category before the filename is searched. Personal files and shared resources have separate canonical locations:

```
/college-files/
|-- students/
|   `-- <cohort>/
|       `-- <department>/
|           `-- <student-id>/
|               |-- assignments/
|               |-- notes/
|               |-- submissions/
|               `-- projects/
|-- projects/
|   `-- <project-id>/
|       |-- shared/
|       |-- deliverables/
|       `-- references/
|-- courses/
|   `-- <course-code>/
|       `-- resources/
`-- administration/
    |-- policies/
    `-- backup-catalog/
```

A project folder has one canonical location under /projects//. A student's projects/ folder can contain a symbolic link to that canonical shared folder. This forms a controlled acyclic-graph view: several student workspaces can point to the same shared project directory without storing uncontrolled copies. The access permissions on the target directory are still enforced when the link is followed.

3.2 Naming rules

Use a predictable filename pattern:

```
<course-code>_<academic-year>_<assignment-or-topic>_<description>_vNN.<ext>
```

Example: CSA0403_2026-27_A4_Task01_FileSystemReview_v01.pdf. Use a controlled character set, avoid spaces where practical, and use a version number for submitted revisions. Student ownership is determined from the protected student-ID directory, not from a student-chosen filename. Names must not contain another student's private identity information.

3.3 File attributes

Maintain the file-system metadata needed to identify, protect, search, audit and recover each file:

Attribute	Purpose
Owner and group	Identify the student owner and the authorised project/course group.
File type and size	Distinguish document/media types and support quota and storage planning.
Timestamps	Record modification time and, where supported, creation/birth and access times.
Permission bits and ACL entries	Enforce read, write, execute and directory-traversal rights for owners, groups and named collaborators.

Attribute	Purpose
Inode/file identifier and link count	Identify the file-system object and help distinguish directory entries from the underlying file.
Checksum/content hash	Support integrity verification and detection of corrupted or changed content.
Version and retention metadata	Support revision history, backup selection and retention/deletion policy.

3.4 Access-control policy

Object	Suggested policy
Personal student directory	Owner-only access by default; POSIX directory mode 0700.
Personal files	Owner read/write by default; POSIX file mode 0600.
Project shared directory	Grant read/write to the project's named group through an ACL; deny access to unrelated students.
Course resources	Read access to the enrolled course group; modification reserved for teaching staff.
Administration and backup catalog	Restricted to authorised staff and backup service accounts.

The server should check permissions on both the directory path and the file. A symbolic link does not bypass permissions on its target. Project membership should be reviewed when a student joins or leaves a group. Administrative access should be logged, and default permissions should be restrictive rather than relying on students to avoid browsing.

3.5 Backup and recovery

Use scheduled, versioned backups with an independent copy on separate storage or a separate site. Apply retention rules for active coursework and completed semesters, test restoration regularly, and retain a backup catalogue containing owner, path, version and restore point. A backup on the same physical disk is not an adequate recovery copy. Shared project folders should be backed up once as canonical content, while the student links are recreated from the directory metadata.

4. Task 1.3 – Design Defence (under 100 words)

Each student's personal directory is private by default through owner-only permissions and ACL checks, so knowing a filename or path does not grant access. Group projects live in a canonical shared directory whose ACL names only authorised members. Symbolic links in student workspaces provide convenient access without creating uncontrolled copies; the target directory still enforces its permissions. Course resources and administrative files have separate group policies. Access changes are logged and membership is reviewed when projects end.

Word count: 78 words.

5. Alternative Concept – Cloud Storage

If student files were stored in a cloud service such as Google Drive or OneDrive, the logical organisation should still separate students, courses and projects, with role- and group-based permissions. The main differences are who operates the storage infrastructure, how users reach files,

and how storage is counted.

Evaluation area	College-managed shared server	Cloud service (e.g., Drive or OneDrive)
Time to find a file	A hierarchical path and indexed directory lookup can make owner-based navigation predictable. Search performance depends on the file system's directory indexing, metadata and server load. Local access avoids a wide-area network round trip for clients on campus.	Provider-side metadata search and filtering can locate files by name, owner, type or other indexed properties. Time to find/open also includes authentication, network latency, provider service load, file size and client caching. It may be convenient for off-campus users, but no fixed faster time can be claimed without measurements.
Storage used, including copies	One canonical file occupies its allocated data blocks; links to it do not necessarily duplicate its content. Additional space is required for backups, snapshots, replicas and separately saved versions.	A shared link may refer to one stored object rather than create another full copy, but uploaded duplicates, version history, provider replication and synced local copies can consume additional space. Exact quota and billing treatment depends on the provider and tenant plan.
College control over privacy	The college directly manages the server, identity integration, ACLs, audit logs, encryption keys, retention, backups and physical location. This provides direct control but also makes the college responsible for security operations and availability.	The college controls tenant identities, sharing policy and available audit/security settings, but the provider operates the underlying infrastructure. Control depends on tenant configuration, contract, encryption/key options, data-location choices, audit features and provider policy. The college must review these before storing sensitive student records.

A cloud service is attractive where off-campus collaboration, remote availability and reduced local server maintenance are priorities. The college must configure external-sharing restrictions, multi-factor authentication, group membership, retention and audit policies. It should also decide whether sensitive records are permitted in the selected tenant and whether a separate export/backup is required for recovery or exit.

6. Recommendation

For the scenario as stated, use a college-managed hierarchical directory structure with unique student-ID paths, private personal directories, ACL-controlled project folders and links to canonical shared project directories. This directly addresses the weaknesses in the proposed flat layout while retaining institutional control over access, retention and backups. Contiguous allocation should not be imposed on every file; allocation should be delegated to a file system suited to file growth. If the college adopts cloud storage, retain the same ownership and group model and verify tenant privacy, retention, audit and data-location controls before migration.

7. Implementation and Submission Notes

Task 1 is a file-system design review. The question does not ask for a C program, a completed TODO block, a shell script or a disk experiment. Therefore, no code implementation or GCC command is required for this task. The evidence expected is the reasoned evaluation, improved organisation and concise privacy-and-sharing justification.